

ESSL Attendance

Installation and User Guide

Attendance management for multiple ESSL / ZKTeco biometric terminals
Node.js · Express · SQLite · zkteco-js

Version 1.0 · August 2026

Contents

1. What this system does

- 1.1 How it works
- 1.2 What you need before starting

2. Installation

- 2.1 Install the application
- 2.2 Development vs production
- 2.3 Install as a service (starts on boot)
- 2.4 Open the firewall

3. First run

- 3.1 Create the admin account
- 3.2 Add your terminals
- 3.3 Map enrollment numbers to employees

4. Daily use

- 4.1 Dashboard
- 4.2 Attendance
- 4.3 Punch log
- 4.4 Exporting to Excel

5. Managing users

- 5.1 Roles
- 5.2 Adding and removing accounts
- 5.3 Passwords

6. How syncing works

- 6.1 Scheduled, manual and live
- 6.2 Why re-syncing never duplicates
- 6.3 Device clocks

7. Troubleshooting

8. Security

9. Maintenance

- 9.1 Backups
- 9.2 Updating
- 9.3 Resetting

10. Reference

- 10.1 Commands
- 10.2 Settings
- 10.3 API endpoints
- 10.4 File layout

1. What this system does

This application collects attendance punches from ESSL / ZKTeco biometric terminals on your network, merges them into one record per employee per day, and presents them in a web page that anyone in the office can open in a browser.

It solves the problems that make multi-device attendance awkward:

- **Three terminals, one employee.** Someone punching in at the Main Gate and out at the Warehouse produces one attendance record, not two.
- **Enrollment numbers are per device.** Employee ID 5 on one terminal is not employee ID 5 on another. The system keeps them separate and lets you link each one to the right person.
- **Re-reading is safe.** A terminal returns its entire log every time it is read. Syncing twice does not create duplicate punches.
- **In/out is worked out for you.** Most staff never press the in/out key, so the first punch of the day is treated as the arrival and the last as the departure.

1.1 How it works

The server connects out to each terminal over your network and reads its stored attendance log. The terminals never connect to the server, so they need no configuration changes and no firewall rules of their own.

```
Browser <---HTTP---> Server (this app) <---ZK protocol---> Terminal 1
                                     |                      Terminal 2
                                     |                      Terminal 3
                                     |
                                   SQLite
```

Everything is stored in a single SQLite file. There is no separate database server to install, and no build step for the web interface.

1.2 What you need before starting

Requirement	Detail
Operating system	Linux with systemd (Ubuntu 22.04 or newer tested)
Node.js	Version 22.5 or newer. The app uses Node's built-in SQLite, so nothing needs compiling.
Network	The server must reach each terminal's IP address, normally on TCP or UDP port 4370.
Terminal details	The IP address of each ESSL device. Read it from the device itself, under its communication or network settings (the exact menu path varies by model).
Permissions	<code>sudo</code> access, needed once to install the boot service.

This installation

Application folder	/home/erpdev/Attendance
Runs as user	erpdev
Server address	<your server IP>
Web address	http://<your server IP>:3000
Node.js	v24.14.0 (installed through nvm)
Database file	/home/erpdev/Attendance/data/attendance.db

2. Installation

2.1 Install the application

From the application folder:

```
cd /home/erpdev/Attendance
npm install
```

That is the whole installation. There is no compiler step, no database server to set up and no web server to configure.

Confirm it works before going near the terminals:

```
npm test
```

Thirteen automated checks run in a few seconds and should all pass. They cover the sync logic and the login system against simulated devices, so a failure here means something is wrong with the installation rather than with your hardware.

2.2 Development vs production

Two environments are provided. Each reads its own settings file and its own database, so experimenting can never damage live attendance data.

	Development	Production
Start with	<code>npm run dev</code>	<code>npm start</code>
Settings file	<code>.env.development</code>	<code>.env.production</code>
Reachable from	this machine only	anywhere on the LAN
Database	<code>data/dev.db</code>	<code>data/attendance.db</code>
Checks devices every	2 minutes	10 minutes
Reloads when files change	yes	no

Development also has a demo mode: `npm run seed:demo` creates three simulated terminals and a week of punches so you can explore every screen without touching real hardware. It only ever writes to the development database.

2.3 Install as a service (starts on boot)

This is the step that makes the system come up on its own whenever the server is switched on, and restart itself if it ever crashes.

```
sudo bash scripts/install-service.sh
```

The script finds your Node.js binary, writes a systemd service file, enables it for boot and starts it. It is safe to run again at any time.

Everyday commands afterwards:

Command	What it does
<code>systemctl status attendance</code>	Is it running?
<code>journalctl -u attendance -f</code>	Watch the live log (Ctrl+C to stop)
<code>sudo systemctl restart attendance</code>	Restart it
<code>sudo systemctl stop attendance</code>	Stop it until the next boot
<code>sudo bash scripts/uninstall-service.sh</code>	Remove the service; data is kept

If Node.js was installed with nvm

systemd needs the exact path to the Node.js binary, and upgrading Node.js through nvm deletes the old folder. When that happens the service stops starting. The fix is to run `sudo bash scripts/install-service.sh` again. To avoid the issue permanently, install Node.js system-wide (at `/usr/bin/node`) and run the installer once more.

If the installer cannot find Node.js, tell it where to look:

```
sudo NODE_BIN="$(which node)" bash scripts/install-service.sh
```

2.4 Open the firewall

If a firewall is active, allow the port — and restrict it to your own network rather than opening it to everything:

```
sudo ufw allow from 192.168.1.0/24 to any port 3000 proto tcp
```

The terminals themselves need no firewall changes, because the server connects out to them rather than the other way round.

3. First run

3.1 Create the admin account

Open `http://<your server IP>:3000`. Because no accounts exist yet, the app shows a **Create the first admin** page. Fill in a username, your name and a password of at least 8 characters.

Do this immediately after installing

Until the first admin exists, that setup page is available to anyone who can reach the server. Once the account is created the page is permanently disabled and everything requires a login. There is no default password to change.

3.2 Add your terminals

Step 1 — Add the device

Go to **Devices** → **Add device**. Fill in:

Field	What to enter
Name	Something recognisable: <i>Main Gate, Warehouse</i>
Location	Optional note about where it is
IP address	The device's address, e.g. <code>192.168.1.201</code>
Port	<code>4370</code> unless it has been changed on the device
Connection	Leave on <i>Auto</i> to begin with
Driver	<i>Real ESSL / ZKTeco device</i>
UDP local port	Leave blank — one is assigned automatically

Step 2 — Test the connection

Press **Test** on the device card. On success it reads back the serial number, model and firmware. If it fails, see *Troubleshooting*.

Step 3 — Sync

Press **Sync now**. This downloads the list of enrolled users and the complete attendance log from that terminal. On a device with months of history the first sync can take a minute.

Repeat for each of your terminals.

3.3 Map enrollment numbers to employees

A terminal only knows enrollment numbers. Before attendance can be reported, each number has to be linked to a person.

Go to the **Mapping** tab. It lists every enrollment number seen on every device, with the name stored on the device where available.

- If your enrollment numbers already match your employee codes, press **Auto-link by matching code** and the whole list links in one action.
- Otherwise pick the right employee from the dropdown beside each number.
- For someone not yet in the system, press **Create employee** to add them and link them in one step.

Linking is retroactive

When you link an enrollment number, every punch already collected under that number is attributed to that employee. You do not need to re-sync.

Remember that the same person enrolled on all three terminals appears three times here — once per device. Link all three to the same employee, and their punches merge into one daily record.

4. Daily use

4.1 Dashboard

The opening screen shows how many staff are present today, how many are absent, the number of punches recorded, and the health of each terminal. Below that is a live feed of the most recent punches. If live capture is switched on for a device, new punches appear here within a second or two without refreshing the page.

4.2 Attendance

Choose a date range and press **Show**. Each row is one employee on one day, merged across every terminal:

Column	Meaning
First in	Earliest punch that day, on any device
Last out	Latest punch that day, on any device
Hours	Time between the two
Punches	How many punches went into the row
Devices	Which terminals the person used
Status	See below

Status	Meaning
Present	Arrived at or before the start of the working day
Late	First punch after the configured start time (09:00 by default)
Incomplete	Only one punch — someone forgot to punch out, or is still at work
Absent	No punches at all that day

Weekends and holidays

The system has no holiday calendar. Every date in the range you choose is counted, so Sundays and public holidays show everyone as *Absent*. This is expected — choose your date ranges accordingly when reporting.

You can narrow the view to one employee or one department, and the summary tiles above the table update to match.

4.3 Punch log

The Attendance tab shows the daily summary; the Punch log shows every individual punch exactly as it came from the terminals, including which device recorded it and whether it arrived by scheduled sync or live capture. Use it when a figure looks wrong and you want to see the raw evidence.

4.4 Exporting to Excel

Both the Attendance and Punch log tabs have an **Export CSV** button. The file downloads with the current filters applied and opens directly in Excel or LibreOffice. Viewer accounts can export too.

5. Managing users

5.1 Roles

Can do	Admin	Viewer
View attendance, punch log, dashboard	Yes	Yes
Export CSV	Yes	Yes
View device status and sync history	Yes	Yes
Add or change devices, sync, set device clocks	Yes	No
Add or change employees, map enrollment numbers	Yes	No
Manage user accounts	Yes	No

Give reception, supervisors and payroll staff a **Viewer** account. They can see and export everything they need without any chance of changing device settings or deleting records.

5.2 Adding and removing accounts

Users → Add user. Choose a username, full name, role and an initial password of at least 8 characters, then pass the password to the person so they can change it.

When someone leaves, either untick **Active** to disable the account or delete it. Disabling is usually better: it ends any session they have open immediately and keeps the record that the account existed.

Sign out on a user's row ends every session that account has open — useful if a laptop is lost or left logged in.

You cannot lock yourself out

The system refuses to demote, disable or delete the last remaining admin. Promote someone else first.

5.3 Passwords

Anyone can change their own password with the **Password** button in the top-right corner. Doing so signs out their other sessions but not the one they are using.

An admin can reset another user's password by editing that user and entering a new one. That immediately signs the person out everywhere.

There is no "forgot password" e-mail

Password resets are done by an admin. If every admin password is lost, recovery means editing the database directly — so make sure at least two people have admin accounts.

6. How syncing works

6.1 Scheduled, manual and live

Method	When it runs
Scheduled	Automatically every 10 minutes in production. This is the reliable baseline and should always stay on.
Manual	Sync all now in the header, or Sync now on one device card.
Live capture	Optional per device. The server holds a connection open and punches appear the moment they happen. A network blip silently ends the stream, which is why the scheduled sync remains as the safety net.

The terminals accept only one connection at a time, so the system queues its own requests per device. Live capture is paused automatically while a sync runs, then resumed. You never need to think about this.

6.2 Why re-syncing never duplicates

Each terminal hands over its complete log on every read, so the same punches arrive again and again. Every punch is stored under a key made of the device serial number, the enrollment number and the exact timestamp, so repeats are recognised and ignored. The Sync log tab shows this: a second sync reports many punches fetched and zero new.

The device serial is used rather than the IP address, so a terminal that picks up a new address from DHCP does not start duplicating its history.

6.3 Device clocks

Terminals report the time from their own internal clock with no timezone attached, so the device and the server must agree. Device clocks drift, especially after a power cut.

Sync clock on a device card sets that terminal's clock from the server. Worth doing when you first install, and any time reported times look shifted.

7. Troubleshooting

The web page will not open

- Check the service is running: `systemctl status attendance`
- Check the log for the reason: `journalctl -u attendance -n 50`
- Confirm you are using the server's address and port 3000, not `localhost`, from another machine.
- If a firewall is on, confirm the rule in section 2.4 was added.

Test fails on a device

1. Confirm the IP is right — read it from the device's own communication/network settings menu.
2. From the server, check it responds at all: `ping 192.168.1.201`
3. If the test times out rather than being refused, edit the device and set **Connection** to **UDP only**. Some ESSL models

answer only on UDP, and automatic fallback cannot detect that case.

4. Confirm nothing else is holding the terminal open — the official ESSL software keeps a connection and the device allows only one.

A device stops working after a while

Usually the terminal has been left in a stuck session. Power-cycle the device, then press **Test**. Check the Sync log tab for the error text recorded at the time.

Punches appear under "unmapped"

Somebody has been enrolled on a terminal but not linked to an employee here. Go to the Mapping tab and link the number. Past punches are attributed automatically.

Someone shows as Absent on a day they worked

- Check the Punch log for that date and device — if there are no punches, the device did not record them.
- Check the Mapping tab: their enrollment number on *that particular terminal* may not be linked, even if the others are.
- Remember that weekends and holidays always show as absent.

Times are wrong by hours

The device clock has drifted or is set to a different timezone. Use **Sync clock** on that device card.

The service does not start after a reboot

Most often Node.js was upgraded through nvm and its folder path changed. Re-run `sudo bash scripts/install-service.sh`. Confirm with `journalctl -u attendance -n 30`.

Everyone is locked out of logging in

Five wrong passwords on one username locks that account for 15 minutes. Wait it out, or restart the service to clear the counter: `sudo systemctl restart attendance`.

8. Security

Access control is enforced on the server, not just hidden in the interface. Every address under `/api` requires a valid session except the login, setup and health checks, and any change (as opposed to a read) requires an admin.

- Passwords are stored using **scrypt** hashing. They are never stored or logged in a readable form, and cannot be recovered from the database — only reset.
- Sessions are held on the server and last 8 hours, extending while in use. Disabling a user, changing their role or resetting their password ends their sessions at once.
- Five failed logins on a username trigger a 15-minute lockout.
- The session cookie cannot be read by page scripts and is not sent by other websites, which protects against the common cross-site attacks.

Traffic is not encrypted

The system runs over plain HTTP. Passwords and session cookies cross your network in readable form, so anyone able to capture network traffic between a browser and the server could read them. This is acceptable on a trusted office network and is not acceptable over the internet.

Never forward port 3000 through your router.

For remote access, use a VPN, or place the application behind a reverse proxy with a TLS certificate.

Keep the firewall rule from section 2.4 in place so only your own network can reach the port.

9. Maintenance

9.1 Backups

Everything — employees, punches, users, device settings — lives in one file. Do not copy it with `cp` while the service is running: the result can be an inconsistent file. Use the supplied script, which takes a proper consistent snapshot and needs

no extra software:

```
bash /home/erpdev/Attendance/scripts/backup.sh /home/erpdev/backups
```

It writes a timestamped file such as `attendance-2026-08-07-2130.db` and reports how many punches it contains. Schedule it daily with `cron`:

```
30 21 * * * bash /home/erpdev/Attendance/scripts/backup.sh /home/erpdev/backups
```

Copy those files somewhere off this server — a failed disk takes the database and the backups with it. To restore, stop the service, put the file back as `data/attendance.db`, and start the service again.

9.2 Updating

```
cd /home/erpdev/Attendance
npm install
sudo systemctl restart attendance
```

If the application files moved or Node.js was upgraded, re-run the service installer as well.

9.3 Resetting

This destroys all attendance history

Deleting the `data` folder permanently removes every punch, employee, user account and device setting. It cannot be undone without a backup. The punches still held in the terminals' own memory can be re-read afterwards, but anything already cleared from the devices is gone for good.

```
sudo systemctl stop attendance
rm -rf /home/erpdev/Attendance/data
sudo systemctl start attendance
```

The database is recreated empty on the next start, and the app returns to the create-first-admin screen.

10. Reference

10.1 Commands

Command	Purpose
<code>npm install</code>	Install dependencies
<code>npm run dev</code>	Start in development mode
<code>npm start</code>	Start in production mode manually
<code>npm run seed:demo</code>	Load demo data into the development database
<code>npm test</code>	Run the automated test suite (13 checks)
<code>bash scripts/backup.sh folder</code>	Take a consistent database backup
<code>npm run docs:build</code>	Rebuild this guide from <code>docs/manual.html</code>
<code>sudo bash scripts/install-service.sh</code>	Install and start the boot service
<code>sudo bash scripts/uninstall-service.sh</code>	Remove the service, keep the data

10.2 Settings

Edit `.env.production` (or `.env.development`), then restart the service.

Setting	Meaning
HOST	0.0.0.0 to allow access from the network, 127.0.0.1 for this machine only
PORT	Web port, default 3000
DB_PATH	Database file location. Use a full path in production.
POLL_INTERVAL_MINUTES	Minutes between automatic syncs; 0 disables them
DEVICE_TIMEOUT_SECONDS	How long to wait for a terminal before giving up
WORKDAY_START	Arrival after this time counts as <i>Late</i>
WORKDAY_END	End of the working day, used for reporting
DUPLICATE_PUNCH_WINDOW_MINUTES	Punches this close together count as one (double-taps on the reader)

10.3 API endpoints

For integrating with payroll or an ERP. All require a session cookie obtained from `POST /api/auth/login`, except where marked public.

Endpoint	Purpose
GET /api/health	Public. Service status.
POST /api/auth/login	Public. Sign in, returns a session cookie.
GET /api/attendance?from&to	Daily attendance records
GET /api/attendance.csv?from&to	The same as a CSV download
GET /api/punches?from&to	Individual punches
GET /api/dashboard	Summary counters
POST /api/sync	Admin. Sync every device now.
GET /api/devices	Devices and their health
GET /api/employees	Employee list
GET /api/events	Live stream of punches (Server-Sent Events)

10.4 File layout

Path	Contents
data/attendance.db	All production data. Back this up.
.env.production	Production settings
.env.development	Development settings
backups/	Where <code>scripts/backup.sh</code> writes by default
scripts/	Service installer, backup script, demo data loader
src/	Application code
public/	Web interface
docs/	This guide
/etc/systemd/system/attendance.service	Boot service definition

Before trusting the first week of data

This guide describes a system that has been tested end to end against simulated terminals, but not against your physical ESSL hardware. For the first few days, spot-check the Punch log against a punch you make yourself on each terminal, and confirm the times and the employee name are right.